

Unit 2: Java Collections Framework

List and ArrayList

ArrayList is a resizable array that implements List. Since Java 8, an ArrayList created with the no-argument constructor starts with an empty internal array; the default capacity of 10 is allocated lazily on the first add. When the array is full it grows by about 50 percent (new capacity = old capacity + old capacity / 2) and the elements are copied. Access by index is $O(1)$; inserting or removing in the middle is $O(n)$ because elements must shift.

LinkedList is a doubly linked list. Adding or removing at either end is $O(1)$, but access by index is $O(n)$. In practice ArrayList is faster for almost all workloads because of CPU cache locality.

HashMap

HashMap stores key-value pairs in an array of buckets. The bucket is chosen from the key's hashCode. The default initial capacity is 16 and the default load factor is 0.75, so the map resizes (doubles) once it holds more than 12 entries. get and put are $O(1)$ on average.

Collisions happen when two keys land in the same bucket. HashMap chains colliding entries in a linked list in that bucket. Since Java 8, when one bucket holds more than 8 entries (TREEIFY_THRESHOLD) and the table has at least 64 buckets, the list is converted into a red-black tree, so the worst case lookup becomes $O(\log n)$ instead of $O(n)$. Keys must implement equals and hashCode consistently: equal objects must have equal hash codes.

HashMap allows one null key and many null values. It is not thread-safe; use ConcurrentHashMap for concurrent access.

TreeMap

TreeMap is a red-black tree that keeps its keys sorted, by natural ordering (Comparable) or by a Comparator passed to the constructor. get, put and remove are $O(\log n)$. TreeMap implements NavigableMap, offering firstKey, lastKey, floorKey, ceilingKey, headMap and tailMap. TreeMap does not allow null keys when using natural ordering.

HashMap vs TreeMap: choose HashMap for the fastest lookups when order does not matter; choose TreeMap when you need keys in sorted order or range queries. LinkedHashMap keeps insertion order with HashMap performance and can act as an LRU cache with accessOrder set to true.

Set

HashSet is backed by a HashMap and does not allow duplicates. TreeSet is backed by a TreeMap and keeps elements sorted. LinkedHashSet preserves insertion order.

Immutable collections

List.of, Set.of and Map.of (Java 9) create unmodifiable collections. They reject null elements and throw UnsupportedOperationException on modification. List.copyOf creates an unmodifiable copy of an existing collection.